

Self-Driving Dynamic Routing Using OSPF and BGP in the BIRD Routing Stack

Contents

1	Abstract	6
2	Project Overview	6
3	Proposal Requirement Mapping	6
4	Lab Topology	8
4.1	Objective	8
4.2	Topology Overview	8
4.3	Router Roles	8
4.4	Host Networks	9
4.5	Routing Design	9
4.6	Conclusion	9
5	Tools and Environment	10
6	Baseline Validation	11
6.1	Objective	11
6.2	Validated Conditions	11
6.3	Baseline Connectivity Results	11
6.4	Conclusion	11
7	Configuration Snapshot	12
7.1	Objective	12
7.2	Captured Files	12
7.3	Snapshot Location	12
7.4	Importance	12
7.5	Conclusion	12
8	BFD WAN Edge Failure Detection	13
8.1	Objective	13
8.2	Test Setup	13
8.3	BFD Timer Configuration	13
8.4	Measurement Result	13
8.5	Path Before Failure	14

8.6	Path During Failure	15
8.7	Traffic Behaviour During Failure	15
8.8	Restore Behaviour	15
8.9	Conclusion	15
9	Multi-hop BFD Failure Detection	16
9.1	Purpose	16
9.2	Multi-hop BFD Endpoints	16
9.3	Failure Scenario	16
9.4	Measurement Result	17
9.5	Conclusion	17
10	OSPF Core Link Failure Recovery	17
10.1	Objective	17
10.2	Test Setup	18
10.3	Measurement Method	18
10.4	Measurement Result	18
10.5	Path Change	18
10.6	Convergence Analysis	19
10.7	Traffic Impact	19
10.8	Post-Test Recovery Validation	19
10.9	Conclusion	20
11	OSPF ECMP Failover	20
11.1	Objective	20
11.2	ECMP Route Under Test	20
11.3	Measurement Methods	21
11.4	OSPF-BFD Activation	21
11.5	Comparison Result	21
11.6	Interpretation	21
11.7	Normal-Rate Traffic Result	22
11.8	Stress-Traffic Result	22
11.9	Route Behaviour	22
11.10	Conclusion	23
12	OSPF NSSA Type-7 to Type-5 Translation	23

12.1 Objective	23
12.2 NSSA Area in the Topology	23
12.3 Why NSSA is Important	23
12.4 Test External Route	24
12.5 Route Visibility Result	24
12.6 Type-7 Evidence Inside NSSA	24
12.7 Type-5 Evidence Outside NSSA	25
12.8 Stub Area Containment	25
12.9 Conclusion	26
13 OSPF Area Healing Across ABRs	26
13.1 Objective	26
13.2 Link Failed	26
13.3 Baseline Route Before Failure	27
13.4 Healing Behaviour After Failure	27
13.5 Measurement Results	27
13.6 Final Restored Health	28
13.7 Interpretation	28
13.8 Conclusion	28
14 BGP Graceful Restart, LLGR, and Route Withdrawal Behaviour	29
14.1 Objective	29
14.2 BGP GR and LLGR Configuration Verification	29
14.3 BGP Graceful Restart Forwarding Test	30
14.3.1 Need for Isolation	30
14.3.2 Isolation Method	30
14.3.3 BGP Restart Action	31
14.3.4 GR Forwarding Result	31
14.3.5 GR Interpretation	31
14.4 BGP Route Withdrawal Test	32
14.4.1 Purpose	32
14.4.2 Withdrawal Isolation Setup	32
14.4.3 Withdrawal Action	33
14.4.4 Withdrawal Result	33
14.4.5 Withdrawal Interpretation	34

14.5 Comparison	34
14.6 Conclusion	34
15 BGP Full Daemon Restart Experiment	35
15.1 Purpose	35
15.2 Experiment Setup	35
15.3 Measured Result	35
15.4 Observation	36
15.5 Conclusion	36
16 BGP Peer Flap Recovery	36
16.1 Purpose	36
16.2 Experiment Setup	36
16.3 Measurement Result	37
16.4 Observation	37
16.5 Conclusion	37
17 LLGR Stale Route Timer Experiment	37
17.1 Purpose	37
17.2 Experiment Setup	38
17.3 Measurement Result	38
17.4 Observation	38
17.5 Conclusion	39
18 Packet Capture Validation	39
18.1 Purpose	39
18.2 Captured Protocols	39
18.3 Measurement Result	40
18.4 Observation	40
18.5 Conclusion	40
19 Result Graphs	40
19.1 Purpose	40
19.2 Convergence and Reaction Time Comparison	41
19.3 Packet Loss Comparison	41
19.4 Additional Deliverable Evidence	42

20 Final Lab Validation	42
21 Summary of Measured Results	43
22 To be Completed	43
23 Conclusion	44
24 Screenshot Evidence Appendix	45
24.1 Additional Deliverable Terminal Evidence	54

1 Abstract

This project implements a self-driving dynamic routing lab using the BIRD routing stack. The objective is to demonstrate how modern enterprise networks detect failures, recover automatically, and preserve forwarding during routing control-plane disruptions.

The lab uses OSPF as the intra-domain routing protocol and BGP as the inter-domain/WAN routing protocol. It includes BFD for fast failure detection, OSPF ECMP for path redundancy, NSSA and stub areas for route containment, OSPF area healing across ABRs, and BGP Graceful Restart / Long-Lived Graceful Restart configuration for forwarding continuity during BGP restarts.

The implementation was validated using a multi-router container-based topology. Each experiment includes configuration evidence, measurement logs, packet-loss results, route-table observations, and screenshots. The final validation confirmed that all end-to-end connectivity paths were restored successfully after the experiments.

2 Project Overview

The project proposal required the design and implementation of a self-driving dynamic routing network capable of detecting failures, automatically choosing alternate paths, reducing brownouts, and demonstrating how OSPF and BGP work together in a resilient network.

The implemented lab satisfies this goal using a realistic enterprise-style topology with:

- OSPF Area 0 as the core backbone.
- Area Border Routers connecting Area 0 to other areas.
- NSSA area for controlled external route redistribution.
- Stub area for branch-style containment.
- eBGP sessions toward an upstream ISP router.
- iBGP between internal WAN edge routers.
- BFD attached to OSPF and BGP sessions.
- BGP Graceful Restart and Long-Lived Graceful Restart configuration.

The project does not simply configure routing protocols. It measures actual behaviour during failures and validates whether the network recovers automatically.

3 Proposal Requirement Mapping

Proposal Requirement	Status	Implemented Evidence
BIRD-based routing lab	Completed	9-router, 3-host BIRD lab topology.
OSPF Area 0 core	Completed	Core routers configured in Area 0.
ABRs to building/branch areas	Completed	ABR behaviour implemented using hpe-r3 and hpe-r4.
NSSA area	Completed	Area 10 configured as NSSA.
Stub area	Completed	Area 20 configured as stub area.
eBGP to ISP/upstream	Completed	hpe-r1/hpe-r2 peer with hpe-r9.
iBGP internally	Completed	hpe-r1 and hpe-r2 internal BGP session.
BFD attached to BGP	Completed	BGP WAN edge failure detected using BFD.
BFD attached to OSPF	Completed	OSPF-BFD sessions enabled and tested.
BFD single-hop detection below 300 ms	Completed	BFD WAN detection measured at 61 ms.
OSPF failure recovery	Completed	OSPF core link failure recovery measured.
OSPF ECMP failover	Completed	Native OSPF ECMP and OSPF-BFD assisted ECMP measured.
NSSA Type-7 / Type-5 behaviour	Completed	Type-7 inside NSSA and Type-5 outside NSSA observed.
OSPF area healing across ABRs	Completed	Area 10 to Area 20 recovery tested after ABR link failure.
BGP Graceful Restart	Completed	Forwarding continuity observed during BGP control-plane restart.
LLGR configuration	Completed	Long-lived graceful restart configured with LL stale time.
Route withdrawal behaviour	Completed	Real route withdrawal caused route removal and packet loss.
Topology diagram	Completed	Topology documented in Markdown, Mermaid, and LaTeX.
Before/after configurations	Completed	Config snapshots and modified configs stored.
Measurement logs	Completed	CSVs, evidence logs, route samples, and ping logs saved.
Screenshots	Completed	Screenshots collected for all major experiments.
Graphs	Pending / Optional	CSV results are available; graphs can be generated as final polish.
BIRD source-code patch	Not in scope	This was an excellence-level optional item, not required for minimum delivery.

4 Lab Topology

4.1 Objective

The lab topology was designed to emulate a small enterprise network using the BIRD routing stack. The topology includes an OSPF campus core, non-backbone OSPF areas, WAN edge routers, an upstream ISP router, internal hosts, eBGP, iBGP, BFD, and BGP Graceful Restart mechanisms.

The purpose of this topology is to provide a realistic environment for testing self-healing routing behaviour under link failures, control-plane restarts, and routing convergence events.

4.2 Topology Overview

The topology contains nine routers and three end hosts. Routers hpe-r1, hpe-r2, hpe-r3, and hpe-r4 form the enterprise OSPF Area 0 core. Routers hpe-r3 and hpe-r4 act as Area Border Routers. Router hpe-r3 connects the core to the NSSA-style Area 10, while router hpe-r4 connects the core to the stub-style Area 20.

The external ISP or upstream network is represented by hpe-r9. The enterprise edge routers hpe-r1 and hpe-r2 connect to hpe-r9 using eBGP. The two enterprise edge routers also exchange routes internally using iBGP.

4.3 Router Roles

Node	Role
hpe-r1	Enterprise WAN edge router, OSPF Area 0 router, eBGP peer with hpe-r9, and iBGP peer with hpe-r2.
hpe-r2	Enterprise WAN edge router, OSPF Area 0 router, eBGP peer with hpe-r9, and iBGP peer with hpe-r1.
hpe-r3	Area Border Router between OSPF Area 0 and NSSA Area 10.
hpe-r4	Area Border Router between OSPF Area 0 and Stub Area 20.
hpe-r5	Internal router inside NSSA Area 10.
hpe-r6	NSSA branch gateway router for host hpe-h1.
hpe-r7	Internal router inside Stub Area 20.
hpe-r8	Stub branch gateway router for host hpe-h2.
hpe-r9	External ISP/upstream router in AS65002 and gateway for hpe-h3.

Table 2: Router roles in the BIRD routing lab

4.4 Host Networks

Host	IP Address	Gateway Router
hpe-h1	10.0.61.2/24	hpe-r6
hpe-h2	10.0.82.2/24	hpe-r8
hpe-h3	10.0.93.2/24	hpe-r9

Table 3: End-host addressing used in the lab

4.5 Routing Design

OSPF is used as the internal routing protocol. Area 0 represents the campus or head-quarters core. Area 10 is used as an NSSA-style branch or guest area, and Area 20 is used as a stub-style branch area. This allows the lab to demonstrate area-based containment and ABR-based healing.

BGP is used for WAN and inter-domain routing. Routers hpe-r1 and hpe-r2 peer with hpe-r9 using eBGP, while hpe-r1 and hpe-r2 also exchange routes internally using iBGP. BGP Graceful Restart and Long-Lived Graceful Restart are configured for later control-plane recovery experiments.

BFD is enabled on WAN-edge BGP sessions to provide fast failure detection. This allows the lab to test sub-second detection and recovery behaviour on edge links.

4.6 Conclusion

The topology provides a realistic small-scale enterprise routing environment. It contains an OSPF core, ABRs, NSSA and stub areas, eBGP WAN connectivity, iBGP internal routing, BFD-based fast failure detection, and BGP state-preservation mechanisms. This makes it suitable for testing the self-healing routing mechanisms required by the project.

Implemented BIRD Routing Lab Topology

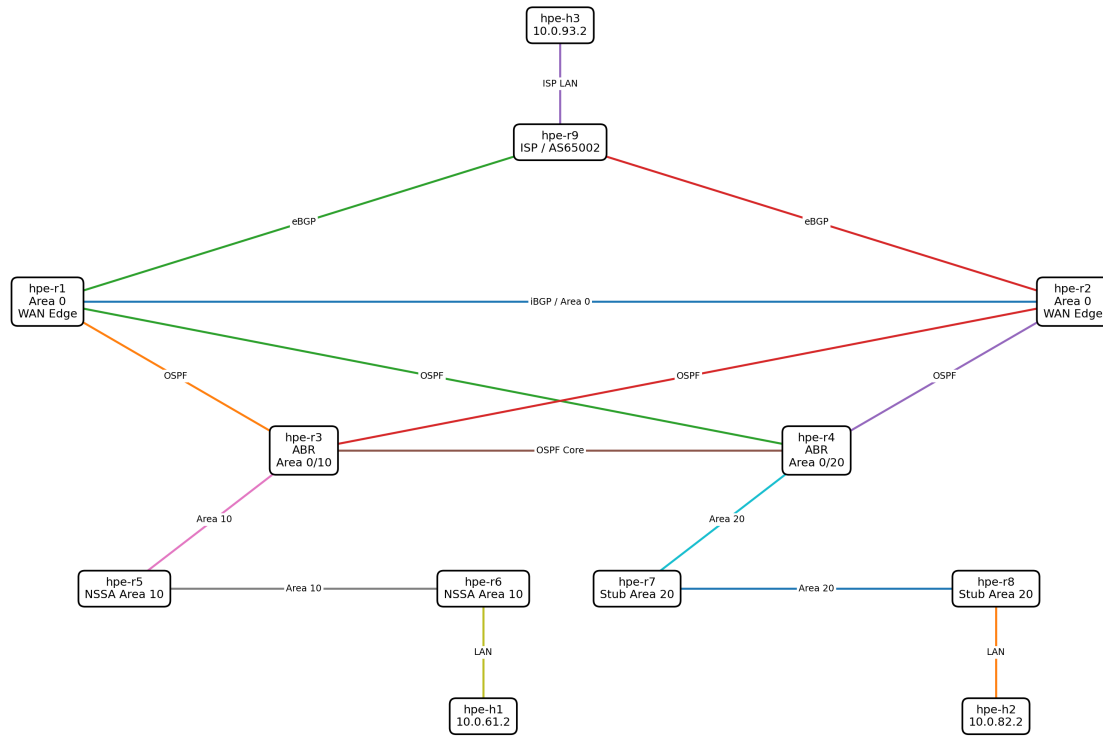


Figure 1: Implemented BIRD routing lab topology

5 Tools and Environment

The project was implemented using a Linux-based container lab.

The main tools used were:

- Linux networking namespaces / containers
- BIRD routing daemon
- BFD inside BIRD
- OSPF and BGP protocol configuration
- Bash scripts for automated experiments
- Python scripts for parsing logs and generating CSV results
- ping for traffic continuity measurement
- ip route and birdc commands for route observation
- Mermaid / LaTeX for documentation

The lab contains routers **hpe-r1** to **hpe-r9** and hosts **hpe-h1**, **hpe-h2**, and **hpe-h3**.

6 Baseline Validation

6.1 Objective

The objective of the baseline validation was to confirm that the routing lab was healthy before running failure experiments.

A stable baseline is important because convergence and failover results are meaningful only when the network starts from a known working state.

6.2 Validated Conditions

The baseline validation checked the following conditions:

- All HPE router and host containers were running.
- BIRD was running on all routers.
- OSPF was running.
- OSPF neighbours were in the Full state.
- BGP sessions were Established.
- BFD sessions on the WAN edge were Up.
- End-to-end host connectivity worked with zero packet loss.

6.3 Baseline Connectivity Results

Test	Source	Destination	Packet Loss
h1_to_h2	hpe-h1	10.0.82.2	0%
h2_to_h1	hpe-h2	10.0.61.2	0%
h1_to_h3	hpe-h1	10.0.93.2	0%
h2_to_h3	hpe-h2	10.0.93.2	0%
h3_to_h1	hpe-h3	10.0.61.2	0%
h3_to_h2	hpe-h3	10.0.82.2	0%

Table 4: Baseline end-to-end connectivity results

6.4 Conclusion

The baseline validation confirmed that the lab was stable before failure testing. OSPF was running with Full neighbour adjacencies, BGP sessions were Established, BFD sessions were Up on the WAN edge, and all host-to-host connectivity tests completed with zero packet loss.

This baseline is the reference state for later BFD, OSPF, ECMP, and BGP GR/LLGR experiments.

7 Configuration Snapshot

7.1 Objective

The objective of this deliverable was to save a clean baseline snapshot of the working lab configuration.

This snapshot was taken after baseline validation confirmed that containers were running, OSPF neighbours were in the Full state, BGP sessions were Established, BFD sessions were Up, and end-to-end host connectivity worked with zero packet loss.

7.2 Captured Files

The snapshot includes the following files:

- BIRD configuration from each router.
- Runtime state from each router.
- Runtime state from each host.
- A manifest describing the snapshot contents.

7.3 Snapshot Location

The timestamped baseline snapshot is stored in:

```
configs/baseline/20260623_142648/
```

The latest baseline snapshot is also available in:

```
configs/baseline/latest/
```

7.4 Importance

The project proposal requires configuration evidence before and after tuning. This snapshot acts as the baseline configuration state before additional experiments such as BFD tuning, OSPF healing, ECMP failover, and BGP GR/LLGR testing.

7.5 Conclusion

The baseline configuration snapshot was successfully created. It provides a reproducible reference point for comparing future tuned configurations and experiment results.

8 BFD WAN Edge Failure Detection

8.1 Objective

The objective of this experiment was to validate fast failure detection on the WAN edge using Bidirectional Forwarding Detection, or BFD.

The tested failure was the WAN edge link between **hpe-r2** and **hpe-r9**. This link carries eBGP between the enterprise edge router and the upstream ISP router. BFD was attached to the BGP session so that link or session failure could be detected quickly.

8.2 Test Setup

Item	Value
Failed router	hpe-r2
Failed interface	eth1
Failed peer	hpe-r9
BFD peer IP	10.0.29.3
Traffic source	hpe-h1
Traffic destination	hpe-h3
Destination IP	10.0.93.2

Table 5: BFD WAN edge failure test setup

8.3 BFD Timer Configuration

The observed BFD session timing values were:

Field	Value
BFD interval	0.100 seconds
BFD timeout	0.500 seconds

Table 6: Observed BFD timing values

8.4 Measurement Result

The BFD WAN edge failure experiment measured how quickly the network detected the failure of the direct WAN edge path between **hpe-r2** and **hpe-r9**. The failed BFD peer was 10.0.29.3. During the test, traffic was continuously sent from **hpe-h1** to **hpe-h3**.

Measurement	Result
Failed router	hpe-r2
Failed interface	eth1
Failed BFD peer	10.0.29.3
Traffic source	hpe-h1
Traffic destination	hpe-h3
Destination IP	10.0.93.2
BFD detection time	61 ms
BGP reaction time	61 ms
Packets transmitted	293
Packets received	293
Packets lost	0
Packet loss	0.00%
Alternate path after failure	hpe-r2 -> hpe-r1 -> hpe-r9

Table 7: BFD WAN edge failure detection measurement result

The project target for single-hop BFD failure detection was below 300 ms. The measured detection time was 61 ms, so the experiment successfully satisfies the target. The ping result also shows that traffic continued with 0.00% packet loss.

Metric	Result
BFD detection time	61 ms
BGP reaction time	61 ms
Estimated packets transmitted	293
Packets received	293
Lost packets	0
Packet loss	0.00%
Explicit unreachable packets	0

Table 8: BFD WAN edge failure measurement result

8.5 Path Before Failure

Before the failure, **hpe-r2** reached the external host network through the direct WAN edge link to **hpe-r9**.

`10.0.93.2 via 10.0.29.3 dev eth1`

This means the original forwarding path was:

`hpe-r2 -> hpe-r9`

8.6 Path During Failure

After the **hpe-r2** to **hpe-r9** link was brought down, BFD detected the failure and BGP removed the failed direct route. During the failure, **hpe-r2** changed its route to the external network through **hpe-r1**.

```
10.0.93.2 via 10.0.12.2 dev eth2
```

At the same time, **hpe-r1** still had reachability to the ISP router through its own WAN edge link.

```
10.0.93.2 via 10.0.19.3 dev eth0
```

Therefore, the alternate forwarding path became:

```
hpe-r2 -> hpe-r1 -> hpe-r9
```

8.7 Traffic Behaviour During Failure

During the failure window, traffic from **hpe-h1** to **hpe-h3** remained successful.

```
10 packets transmitted, 10 received, 0\% packet loss
```

The longer timestamped ping monitor also showed:

```
293 packets transmitted, 293 received, 0 lost, 0.00\% packet loss
```

8.8 Restore Behaviour

After the failed link was restored, BFD became Up again on both sides. The BGP session between **hpe-r2** and **hpe-r9** also returned to Established state after a short reconnection period.

The final restore confirmation showed that direct link pings worked, BFD was Up, BGP was Established, and end-to-end traffic from **hpe-h1** to **hpe-h3** worked with zero packet loss.

8.9 Conclusion

The BFD WAN edge failure experiment successfully demonstrated fast failure detection and automatic routing recovery.

BFD detected the **hpe-r2** to **hpe-r9** WAN edge failure in 61 ms. BGP reacted in the same 61 ms window. After the failed direct path was removed, traffic shifted to the alternate path through **hpe-r1** and continued toward **hpe-r9**.

The traffic test showed 0.00% packet loss during the failure window. After link restoration, BFD and BGP recovered cleanly. This confirms that BFD can provide sub-second WAN failure detection and help maintain reachability through alternate paths.

9 Multi-hop BFD Failure Detection

9.1 Purpose

The earlier BFD experiments used directly connected BFD peers. In this experiment, a multi-hop BFD session was configured between **hpe-r1** and **hpe-r8**. These routers are not directly connected, so BFD packets travel through the routed OSPF network.

The goal of this experiment was to verify whether BFD can detect failure of a routed multi-hop path within the project target of less than 1 second.

9.2 Multi-hop BFD Endpoints

The multi-hop BFD session used the following endpoints:

Router	Multi-hop BFD Peer
hpe-r1	10.0.82.3
hpe-r8	10.0.14.2

Table 9: Multi-hop BFD endpoints

Before failure, the BFD session was observed in the **Up** state on both routers. The interface column showed **---**, confirming that the session was not tied to a directly connected interface.

9.3 Failure Scenario

The routed path between the two BFD endpoints used the branch-side path through **hpe-r4** and **hpe-r7**. To trigger a multi-hop reachability failure, the interface **eth2** on **hpe-r4** was brought down.

```
docker exec hpe-r4 ip link set eth2 down
```

This broke the path:

```
hpe-r1 -> hpe-r4 -> hpe-r7 -> hpe-r8
```

9.4 Measurement Result

Measurement	Result
Failed link	<code>hpe-r4 eth2</code> to <code>hpe-r7 eth1</code>
<code>hpe-r1</code> BFD detection time	546 ms
<code>hpe-r8</code> BFD detection time	629 ms
<code>hpe-r1</code> BFD recovery time	7645 ms
<code>hpe-r8</code> BFD recovery time	2 ms
Ping packets transmitted	269
Ping packets received	79
Ping loss	70.632%
Target below 1 second	Yes

Table 10: Multi-hop BFD failure detection result

The measured multi-hop BFD failure detection times were 546 ms on `hpe-r1` and 629 ms on `hpe-r8`. Both values are below the 1-second target.

The packet loss is high because the experiment intentionally broke the routed path between the multi-hop BFD endpoints. This test mainly validates fast multi-hop failure detection, not traffic failover.

9.5 Conclusion

This experiment satisfies the multi-hop BFD deliverable. It proves that BIRD can run BFD between non-directly connected routers and detect routed-path failure in sub-second time.

10 OSPF Core Link Failure Recovery

10.1 Objective

The objective of this experiment was to validate OSPF fast recovery when an internal core link fails.

The tested failure was the core OSPF link between `hpe-r3` and `hpe-r2`. The goal was to observe whether OSPF could detect the failure, remove the failed path, install an alternate path, and restore forwarding quickly.

Item	Value
Failed router	hpe-r3
Failed core peer	hpe-r2
Old next-hop	10.0.23.2
New next-hop	10.0.13.2
Test type	OSPF core link failure
Primary measurement method	Linux kernel route monitor
Supporting methods	ip route get and BIRD route sampling
Traffic measurement	Timestamped ICMP ping

Table 11: OSPF core link failure test setup

10.2 Test Setup

10.3 Measurement Method

OSPF convergence is not only a control-plane event. A route may change inside BIRD first, but traffic is forwarded by the Linux kernel routing table.

Because of this, three different measurement methods were used. The Linux kernel route monitor was treated as the primary convergence measurement because it shows when the actual forwarding route changed in the kernel. The `ip route get` and BIRD route-table samples were used as supporting confirmation.

10.4 Measurement Result

Metric	Result
Old next-hop	10.0.23.2
New next-hop	10.0.13.2
Kernel route monitor convergence time	628 ms
ip route get sample convergence time	631 ms
BIRD route-table sample convergence time	648 ms
Estimated transmitted packets	350
Received packets	338
Failed/lost packets	12
Estimated packet loss	3.43%
Explicit unreachable packets	1

Table 12: OSPF core link failure measurement result

10.5 Path Change

Before the failure, the route used the old next-hop:

10.0.23.2

After the core link failure, OSPF selected the alternate next-hop:

```
10.0.13.2
```

This means OSPF successfully removed the failed path and installed an alternate route.

10.6 Convergence Analysis

The three convergence measurements were close:

```
Kernel route monitor: 628 ms
ip route get sampling: 631 ms
BIRD route sampling: 648 ms
```

This consistency shows that the result is reliable. The kernel forwarding table changed first at around 628 ms, and the supporting measurements confirmed the same route transition within a small time gap.

10.7 Traffic Impact

The ping log did not contain a normal final ping summary, so packet loss was estimated using ICMP sequence numbers.

The parsed result was:

```
Estimated transmitted packets: 350
Received packets: 338
Failed/lost packets: 12
Estimated packet loss percent: 3.43%
Explicit unreachable packets seen: 1
```

This shows that traffic experienced a short disruption during OSPF reconvergence, but connectivity recovered automatically after the alternate path was installed.

10.8 Post-Test Recovery Validation

After the failure experiment, OSPF was verified on all internal routers. OSPF was Running on hpe-r1 through hpe-r8.

Final end-to-end pings also succeeded with zero packet loss:

```
hpe-h1 -> hpe-h2: 0% packet loss
hpe-h1 -> hpe-h3: 0% packet loss
hpe-h3 -> hpe-h1: 0% packet loss
```

10.9 Conclusion

The OSPF core link failure experiment successfully demonstrated automatic intra-domain recovery.

When the `hpe-r3` to `hpe-r2` core link failed, OSPF removed the old path through `10.0.23.2` and installed the alternate path through `10.0.13.2`.

The primary convergence time measured using the Linux kernel route monitor was 628 ms. Supporting measurements using `ip route get` and BIRD route-table sampling showed 631 ms and 648 ms respectively. Since all three values were close, the convergence measurement is consistent and reliable.

The traffic test showed an estimated packet loss of 3.43% during the short convergence window. After reconvergence, OSPF was Running on all internal routers and end-to-end connectivity returned to zero packet loss.

11 OSPF ECMP Failover

11.1 Objective

The objective of this experiment was to validate OSPF Equal-Cost Multi-Path, or ECMP, failover.

The experiment checked whether a router with two equal-cost OSPF next-hops could continue forwarding traffic when one ECMP branch failed. The experiment also compared native OSPF ECMP behaviour with BFD-assisted OSPF ECMP behaviour.

11.2 ECMP Route Under Test

The ECMP route was observed on `hpe-r3`.

```
10.0.24.0/24
via 10.0.23.2 on eth3 weight 1
via 10.0.34.3 on eth0 weight 1
```

The same route was also installed in the Linux kernel as a multipath route.

```
10.0.24.0/24 proto bird metric 32
nexthop via 10.0.23.2 dev eth3 weight 1
nexthop via 10.0.34.3 dev eth0 weight 1
```

This proves that `hpe-r3` had two equal-cost OSPF forwarding branches:

```
hpe-r3 -> hpe-r2
hpe-r3 -> hpe-r4
```

The failed ECMP next-hop was `10.0.23.2`. The surviving ECMP next-hop was `10.0.34.3`.

11.3 Measurement Methods

Multiple measurement methods were used because ECMP failover has both control-plane and data-plane behaviour.

The Linux kernel route monitor detected when the kernel forwarding route changed. Kernel route sampling confirmed when the failed ECMP next-hop was removed from the kernel route. The `ip route get` sampler showed the selected forwarding next-hop for the test destination. BIRD route-table sampling confirmed when BIRD removed the failed next-hop. ICMP ping measured packet loss during the failover window.

11.4 OSPF-BFD Activation

Initially, ECMP failover was measured using native OSPF behaviour. Then BFD was enabled on OSPF interfaces.

After enabling BFD, `hpe-r3` showed active BFD sessions to both ECMP neighbours:

```
10.0.23.2 eth3 Up
10.0.34.3 eth0 Up
```

This confirms that BFD was active on the OSPF ECMP core links.

11.5 Comparison Result

Test	BFD	Ping	Kernel	Route Sample	BIRD	Loss
Native OSPF ECMP	No	0.02s	840 ms	820 ms	819 ms	16.00%
OSPF ECMP with BFD, stress	Yes	0.02s	292 ms	305 ms	304 ms	7.40%
OSPF ECMP with BFD, normal	Yes	1s	517 ms	542 ms	540 ms	1 packet

Table 13: OSPF ECMP failover comparison

11.6 Interpretation

The native OSPF ECMP baseline showed that the failed ECMP next-hop was removed, but convergence took longer.

After enabling BFD on OSPF interfaces, the ECMP failover improved significantly in the high-rate stress test.

Kernel monitor time improved from 840 ms to 292 ms.

BIRD route-table update improved from 819 ms to 304 ms.

Packet loss reduced from 16.00\% to 7.40\%.

This shows that BFD helped OSPF detect the failed ECMP branch faster and remove the failed next-hop more quickly.

11.7 Normal-Rate Traffic Result

The normal-rate ping test used one ICMP packet per second.

```
20 packets transmitted
19 received
1 packet lost
5.00\% packet loss
```

Although the percentage appears as 5%, this represents only one lost packet during the failover window. This is a practical way to describe normal-rate traffic impact.

11.8 Stress-Traffic Result

The high-rate stress test used one packet every 0.02 seconds, or around 50 packets per second.

In the BFD-assisted stress test:

```
500 packets transmitted
463 received
37 packets lost
7.40\% packet loss
```

At 50 packets per second, 37 lost packets corresponds to a short disruption window. This stress test made the failover window more visible.

11.9 Route Behaviour

Before failure, both ECMP next-hops were present:

```
nexthop via 10.0.23.2 dev eth3 weight 1
nexthop via 10.0.34.3 dev eth0 weight 1
```

After the failure of the 10.0.23.2 branch, the kernel route changed to the surviving next-hop:

```
10.0.24.0/24 via 10.0.34.3 dev eth0 proto bird metric 32
```

This proves that the failed ECMP branch was removed and forwarding continued through the surviving branch.

11.10 Conclusion

The OSPF ECMP failover experiment successfully demonstrated equal-cost multipath routing and failover.

Before failure, `hpe-r3` had two equal-cost next-hops toward `10.0.24.0/24`: `10.0.23.2` and `10.0.34.3`. When the `10.0.23.2` branch failed, OSPF removed the failed next-hop and continued forwarding using the surviving next-hop `10.0.34.3`.

The native OSPF ECMP baseline converged in 840 ms by kernel route monitor measurement. After BFD was enabled on OSPF interfaces, the stress-test convergence improved to 292 ms, and packet loss reduced from 16.00% to 7.40%.

Under normal-rate traffic, only one packet was lost during ECMP failover. This confirms that OSPF ECMP failover worked correctly, and BFD-assisted OSPF improved failure detection and reduced traffic disruption.

12 OSPF NSSA Type-7 to Type-5 Translation

12.1 Objective

The objective of this experiment was to validate OSPF NSSA behaviour, specifically Type-7 to Type-5 external route translation and external route containment.

The experiment checked whether an external route originated inside an NSSA area is first carried as a Type-7 LSA inside the NSSA, and then translated into a Type-5 LSA by the Area Border Router, or ABR, for the backbone area.

12.2 NSSA Area in the Topology

In this topology, Area 10 is configured as an NSSA area.

The relevant routers are:

Router	Role
<code>hpe-r3</code>	ABR between Area 0 and Area 10
<code>hpe-r5</code>	Internal NSSA router
<code>hpe-r6</code>	Internal NSSA router and origin router for the test external route

Table 14: Routers involved in the NSSA experiment

Area 10 is configured as NSSA on `hpe-r3`, `hpe-r5`, and `hpe-r6`.

12.3 Why NSSA is Important

In normal OSPF, external routes are flooded as Type-5 LSAs.

However, an NSSA does not directly flood Type-5 LSAs inside the NSSA area. Instead, external routes originated inside the NSSA are carried as Type-7 LSAs. The ABR then

translates the Type-7 LSA into a Type-5 LSA for the rest of the OSPF domain.

This keeps external route flooding controlled and prevents external LSAs from entering areas where they should not be directly flooded.

12.4 Test External Route

A controlled static blackhole route was added on **hpe-r6**.

```
172.16.66.0/24
```

This was added as a BIRD static route:

```
172.16.66.0/24 blackhole
```

This route was then exported into OSPF from **hpe-r6**, which is inside NSSA Area 10.

The route was intentionally configured as a blackhole route because the experiment only needed a clean routing advertisement, not real host connectivity.

12.5 Route Visibility Result

The route visibility result is shown below.

Router	Route Present	Route Type
hpe-r6	Yes	Static
hpe-r5	Yes	OSPF-E2
hpe-r3	Yes	OSPF-E2
hpe-r1	Yes	OSPF-E2
hpe-r2	Yes	OSPF-E2
hpe-r4	Yes	OSPF-E2
hpe-r8	No	Not present

Table 15: NSSA external route visibility

This confirms that the test route was originated by **hpe-r6**, propagated through Area 10, translated by the ABR, and made visible to the backbone routers.

12.6 Type-7 Evidence Inside NSSA

The Type-7 LSADB output showed the test route inside the NSSA area.

```
hpe-r6:  
0007 172.16.66.255 6.6.6.6
```

```
hpe-r5:
```

```
0007 172.16.66.255 6.6.6.6
```

hpe-r3:

```
0007 172.16.66.255 6.6.6.6
```

This proves that the external route originated by **hpe-r6** was carried inside Area 10 as a Type-7 LSA.

12.7 Type-5 Evidence Outside NSSA

The Type-5 LSADB output showed the translated route outside the NSSA.

hpe-r3:

```
0005 172.16.66.255 3.3.3.3
```

hpe-r1:

```
0005 172.16.66.255 3.3.3.3
```

hpe-r2:

```
0005 172.16.66.255 3.3.3.3
```

hpe-r4:

```
0005 172.16.66.255 3.3.3.3
```

This proves that ABR **hpe-r3**, whose router ID is 3.3.3.3, translated the NSSA Type-7 LSA into a Type-5 LSA for the backbone area.

12.8 Stub Area Containment

Router **hpe-r8** is inside the stub area, Area 20.

When the route was checked on **hpe-r8**, BIRD reported:

```
Network not found
```

This means **hpe-r8** did not receive the external OSPF route as a direct OSPF external route.

However, the Linux kernel still had a forwarding path using the default route:

```
172.16.66.1 via 10.0.78.2 dev eth0
```

This is the expected containment behaviour. The external route itself was not flooded into the stub area, but the stub router could still forward traffic using the default route.

12.9 Conclusion

The NSSA experiment successfully demonstrated Type-7 to Type-5 translation and external route containment.

A static blackhole route, 172.16.66.0/24, was originated inside NSSA Area 10 from **hpe-r6**. Inside the NSSA, the route appeared as a Type-7 LSA originated by router ID 6.6.6.6. At the ABR, **hpe-r3**, the Type-7 LSA was translated into a Type-5 LSA originated by router ID 3.3.3.3.

Backbone routers such as **hpe-r1**, **hpe-r2**, and **hpe-r4** received the translated Type-5 LSA. The stub-area router **hpe-r8** did not receive the external route directly, showing that external route flooding was contained.

This confirms that NSSA behaviour worked correctly in the lab topology.

13 OSPF Area Healing Across ABRs

13.1 Objective

The objective of this experiment was to validate OSPF area healing across ABRs.

The experiment tested whether traffic between two different OSPF areas could recover when the direct ABR-to-ABR/core link failed.

The traffic tested was:

```
hpe-h1 -> hpe-h2  
10.0.61.2 -> 10.0.82.2
```

In the topology, **hpe-h1** is behind Area 10, and **hpe-h2** is behind Area 20. Router **hpe-r3** acts as the ABR between Area 10 and Area 0, while **hpe-r4** acts as the ABR between Area 20 and Area 0.

13.2 Link Failed

The failed link was the direct core/ABR link between **hpe-r3** and **hpe-r4**.

The failed interface was:

```
hpe-r3 eth0
```

Before the failure, **hpe-r3** reached the Area 20 host network through **hpe-r4**.

```
10.0.82.0/24  
via 10.0.34.3 on eth0
```

The old direct next-hop was:

```
10.0.34.3
```

13.3 Baseline Route Before Failure

Before the failure, the route from `hpe-r3` toward `10.0.82.0/24` was:

```
10.0.82.0/24 unicast [ospf1] IA
via 10.0.34.3 on eth0
```

The Linux kernel forwarding decision also used the same next-hop:

```
10.0.82.2 via 10.0.34.3 dev eth0
```

This proves that before failure, Area 10 to Area 20 traffic used the direct `hpe-r3 -> hpe-r4` path.

13.4 Healing Behaviour After Failure

After `hpe-r3 eth0` was brought down, OSPF recalculated an alternate path.

The route-get sampler first showed the new forwarding path as:

```
10.0.82.2 via 10.0.23.2 dev eth3
```

The Linux kernel route later showed two alternate next-hops:

```
10.0.82.0/24 proto bird metric 32
nexthop via 10.0.13.2 dev eth2 weight 1
nexthop via 10.0.23.2 dev eth3 weight 1
```

The BIRD route table also showed the healed route:

```
10.0.82.0/24 unicast [ospf1] IA
via 10.0.13.2 on eth2 weight 1
via 10.0.23.2 on eth3 weight 1
```

This proves that after the direct ABR link failed, traffic healed through alternate back-bone paths via `hpe-r1` and `hpe-r2`.

13.5 Measurement Results

The missing ICMP sequence numbers were:

```
11 12 13 14 15 16 17 18 19 20 21 22 23 24
```

The ping interval used during the test was 0.1 seconds. This means the 14 lost packets represent a short disruption window of around 1.4 seconds during OSPF area healing.

Measurement	Result
Route-get switch time	80 ms
Kernel route switch time	931 ms
BIRD route switch time	962 ms
Ping packets transmitted	150
Ping packets received	136
Ping packets lost	14
Packet loss	9.33333%

Table 16: OSPF area healing measurements

13.6 Final Restored Health

After the failed link was restored, connectivity was checked again.

```
5 packets transmitted
5 received
0\% packet loss
```

This confirms that the network returned to a healthy state after the link was restored.

13.7 Interpretation

The experiment shows three important things.

First, the direct ABR path between **hpe-r3** and **hpe-r4** was being used before the failure.

Second, when that direct link failed, OSPF did not leave Area 10 to Area 20 traffic broken. It recalculated alternate Area 0 backbone paths through **hpe-r1** and **hpe-r2**.

Third, after the link was restored, normal connectivity returned with 0% packet loss.

The route-get result changed quickly, within 80 ms. The full kernel and BIRD route-table convergence took around 1 second. During this convergence window, 14 out of 150 high-frequency ping packets were lost.

13.8 Conclusion

The OSPF area healing experiment successfully demonstrated inter-area recovery across ABRs.

Before failure, traffic from Area 10 toward Area 20 used the direct path from **hpe-r3** to **hpe-r4** through next-hop 10.0.34.3. After the **hpe-r3 eth0** link failed, OSPF recalculated the path and installed alternate next-hops through 10.0.13.2 and 10.0.23.2.

The route-get sampler detected a forwarding change in 80 ms, while the full kernel and BIRD route-table updates completed in around 1 second. During the transition, 14 packets were lost in a high-frequency ping test, but after restoration the final health check showed 0% packet loss.

This confirms that OSPF area healing worked correctly across ABRs in the lab topology.

14 BGP Graceful Restart, LLGR, and Route Withdrawal Behaviour

14.1 Objective

The objective of this experiment was to test BGP control-plane restart behaviour and route withdrawal behaviour.

This experiment has two parts:

1. BGP Graceful Restart / LLGR forwarding continuity test
2. BGP route withdrawal test

In the Graceful Restart test, the BGP session was restarted, but the route was expected to be retained temporarily so that forwarding could continue.

In the withdrawal test, the route itself was intentionally removed from BGP export. In that case, forwarding should eventually fail because the route is no longer valid.

14.2 BGP GR and LLGR Configuration Verification

Before running the test, the BGP configuration was checked on the BGP routers.

The following BGP options were already configured:

```
graceful restart yes;  
long lived graceful restart yes;
```

The protocol details also showed:

```
Long-lived graceful restart  
LL stale time: 3600  
Restart time: 120  
BGP state: Established
```

The BGP sessions were established before testing.

Router	BGP Peer	State
hpe-r1	hpe-r9	Established
hpe-r1	hpe-r2	Established
hpe-r2	hpe-r9	Established
hpe-r9	hpe-r1	Established
hpe-r9	hpe-r2	Established

Table 17: BGP session state before GR and withdrawal testing

14.3 BGP Graceful Restart Forwarding Test

14.3.1 Need for Isolation

A simple BGP restart test is not enough to prove Graceful Restart. If the `hpe-r1 -> hpe-r9` BGP path fails, traffic might simply move through another path:

```
hpe-r1 -> hpe-r2 -> hpe-r9
```

That would prove BGP failover, not Graceful Restart. Therefore, the final test used isolation.

14.3.2 Isolation Method

The final Graceful Restart proof used a blackhole guard and disabled the alternate path.

The setup was:

```
Traffic: hpe-h1 -> hpe-h3
Source host: 10.0.61.2
Destination host: 10.0.93.2
Target prefix: 10.0.93.0/24
Expected BGP next-hop: 10.0.19.3
```

A blackhole guard route was added on `hpe-r1`:

```
blackhole 10.0.93.0/24 metric 9999
```

At the same time, the valid BGP route still existed:

```
10.0.93.0/24 via 10.0.19.3 dev eth0 proto bird metric 32
```

The Linux route lookup still preferred the real BGP route:

```
10.0.93.2 via 10.0.19.3 dev eth0
```

Then the alternate `hpe-r1 -> hpe-r2` path was disabled:

```
hpe-r1 eth1 DOWN
hpe-r1 protocol r2 disabled/down
```

This means the test had no usable alternate path through `hpe-r2`.

14.3.3 BGP Restart Action

The BGP session from **hpe-r9** toward **hpe-r1** was restarted:

```
docker exec hpe-r9 birdc restart r1
```

During the test, the BGP session on **hpe-r1** temporarily became non-established:

```
BGP non-established seen ms: 114
State: Active
Received: Administrative reset
```

This proves that a real BGP control-plane restart happened.

14.3.4 GR Forwarding Result

Measurement	Result
Blackhole guard enabled	yes
hpe-r1 -> hpe-r2 link down	yes
Old next-hop after restart	yes
Alternate next-hop after restart	no
Blackhole/unreachable after restart	no
BGP non-established seen	114 ms
Ping transmitted	200
Ping received	200
Ping lost	0
Packet loss	0%

Table 18: BGP GR forwarding continuity result

The important result lines were:

```
Old next-hop seen after restart: yes
Alternate next-hop seen after restart: no
Blackhole/unreachable seen after restart: no
Ping transmitted: 200
Ping received: 200
Ping lost: 0
Ping loss percent: 0
```

14.3.5 GR Interpretation

This confirms that forwarding continued during the BGP restart.

The result is strong because the **hpe-r1** -> **hpe-r2** alternate path was disabled, and a blackhole guard was installed for 10.0.93.0/24. The traffic did not fall into the blackhole

route. Instead, route lookup continued to use the old direct next-hop 10.0.19.3, and no ping packets were lost.

Therefore, the traffic continuity was not caused by alternate routing or static/default fallback. It was caused by the forwarding plane continuing to use the retained route during the BGP restart.

The sampled BIRD route output did not show a visible ‘stale’ or ‘LLGR stale’ label. Therefore, the conclusion is not that BIRD visibly marked the route as stale. The correct conclusion is that forwarding continuity was observed during BGP restart under a GR/LLGR-enabled configuration.

14.4 BGP Route Withdrawal Test

14.4.1 Purpose

The withdrawal test proves the opposite behaviour.

In this test, the BGP session stayed up, but the prefix 10.0.93.0/24 was intentionally withdrawn from export toward `hpe-r1`.

The purpose was to show that Graceful Restart protects forwarding during restart, but it does not keep forwarding alive when the route is truly withdrawn.

14.4.2 Withdrawal Isolation Setup

The same isolation idea was used:

```
blackhole guard enabled
hpe-r1 -> hpe-r2 alternate path disabled
```

Before withdrawal, `hpe-r1` still had the valid BGP route:

```
10.0.93.0/24 via 10.0.19.3 dev eth0 proto bird metric 32
blackhole 10.0.93.0/24 metric 9999
```

Route lookup still preferred the real BGP route:

```
10.0.93.2 via 10.0.19.3 dev eth0
```

The alternate path was disabled:

```
hpe-r1 eth1 DOWN
```

A baseline ping before withdrawal succeeded:

```
5 packets transmitted
5 received
0\% packet loss
```

14.4.3 Withdrawal Action

The hpe-r9 BGP export policy toward hpe-r1 was changed to reject the host-side prefix:

10.0.93.0/24

The modified configuration was applied using:

```
docker exec hpe-r9 birdc configure
```

14.4.4 Withdrawal Result

Measurement	Result
Blackhole guard enabled	yes
hpe-r1 -> hpe-r2 path disabled	yes
Withdrawn prefix	10.0.93.0/24
BGP session non-established after withdrawal	no
Alternate next-hop after withdrawal	no
BIRD route missing time	134 ms
Ping transmitted	220
Ping received	11
Ping lost	209
Packet loss	95%
Final ping after withdrawal	100% packet loss

Table 19: BGP route withdrawal result

The important result lines were:

```
BGP non-established seen after withdrawal: no
BIRD route missing ms: 134
BIRD route missing line: Network not found
Ping transmitted: 220
Ping received: 11
Ping lost: 209
Ping loss percent: 95
```

Final ping after withdrawal:

```
5 packets transmitted
0 received
100\% packet loss
```

14.4.5 Withdrawal Interpretation

This proves that the route was actually withdrawn while the BGP session stayed up.

The session did not go down:

```
BGP non-established seen after withdrawal: no
```

But the route disappeared from BIRD:

```
Network not found
```

After the route disappeared, traffic failed.

This is the expected behaviour. Graceful Restart helps during a control-plane restart, but it should not keep forwarding traffic forever for a route that has been intentionally withdrawn.

14.5 Comparison

Test	BGP Session	Route Behaviour	Traffic Result
GR forwarding test	Temporarily Active	Forwarding continued via old next-hop	0% loss
Route withdrawal test	Stayed established	Route became Network not found	95% loss during test

Table 20: Comparison of BGP Graceful Restart and route withdrawal

14.6 Conclusion

The BGP Graceful Restart / LLGR experiment showed that forwarding continued during a BGP control-plane restart. During the restart, the BGP session temporarily moved into a non-established state, but traffic continued through the old direct next-hop 10.0.19.3 with 0% packet loss.

The test was isolated using a blackhole guard and by disabling the alternate **hpe-r1** -> **hpe-r2** path. Therefore, the result was not caused by alternate routing or default-route fallback.

The route withdrawal test showed the opposite behaviour. When 10.0.93.0/24 was intentionally withdrawn from BGP export, the BGP session stayed up, but the route disappeared from BIRD within 134 ms. Traffic then failed, with 95% packet loss during the test and 100% packet loss in the final ping.

Together, these two tests show the difference between Graceful Restart route retention and true route withdrawal.

15 BGP Full Daemon Restart Experiment

15.1 Purpose

The earlier BGP Graceful Restart experiment tested a BGP protocol/control-plane restart. In this additional experiment, the full BIRD daemon on `hpe-r9` was restarted to observe whether forwarding would remain fully preserved during a stronger control-plane disruption.

15.2 Experiment Setup

The restarted router was `hpe-r9`, which acts as the ISP/upstream router. Traffic was continuously sent from `hpe-h1` to `hpe-h3`.

```
hpe-h1 -> hpe-h3
10.0.61.2 -> 10.0.93.2
```

To reduce the chance of BFD immediately tearing down the BGP session, BGP-BFD was temporarily disabled for this controlled test. Graceful Restart and Long-Lived Graceful Restart remained enabled.

A kernel `persist;` statement was also added on `hpe-r9` before the test so that kernel forwarding routes would be less likely to be removed during daemon restart.

15.3 Measured Result

Measurement	Result
Restarted router	<code>hpe-r9</code>
Restart type	Full BIRD daemon crash/restart
BGP-BFD state during test	Temporarily disabled
BIRD control socket ready time	84 ms
BGP re-establishment time	4539 ms
Ping packets transmitted	170
Ping packets received	97
Packet loss	42.9412%
Forward route missing samples	0
Return route missing samples	0
Forwarding fully preserved	No

Table 21: BGP full daemon restart measurement result

15.4 Observation

The full BIRD daemon restart caused visible packet loss even though route-monitor samples did not show missing forward or return routes. This indicates that a full daemon restart is more disruptive than a BGP protocol restart.

The earlier BGP protocol/control-plane restart test preserved forwarding with 0% packet loss. However, restarting the complete BIRD daemon affected more components, including protocol state, kernel synchronization, control socket availability, and route export behaviour.

15.5 Conclusion

This experiment shows an important limitation. Graceful Restart and LLGR were effective for BGP protocol/control-plane restart, but full BIRD daemon restart did not fully preserve end-to-end forwarding in this lab. This means production networks must carefully coordinate GR, LLGR, BFD, and forwarding-plane route persistence during planned daemon maintenance.

16 BGP Peer Flap Recovery

16.1 Purpose

This experiment tests how the network behaves when a BGP peer temporarily flaps. The goal is to verify whether traffic can automatically move to an alternate BGP path and continue without packet loss.

16.2 Experiment Setup

The BGP peer between `hpe-r1` and `hpe-r9` was disabled and then re-enabled.

```
birdc disable r9
birdc enable r9
```

The target traffic was from `hpe-r1` to the external host network behind `hpe-r9`.

```
hpe-r1 -> 10.0.93.2
```

Before the peer flap, `hpe-r1` used the direct next-hop `10.0.19.3`. During the peer flap, it switched to the alternate path through `hpe-r2` using next-hop `10.0.12.3`.

16.3 Measurement Result

Measurement	Result
Flapped router	<code>hpe-r1</code>
Flapped BGP peer	<code>r9</code>
Traffic target	<code>10.0.93.2</code>
Route switched to alternate next-hop	93 ms
BGP peer re-established	1596 ms
Route switched back to direct next-hop	1754 ms
Ping packets transmitted	162
Ping packets received	162
Packet loss	0%

Table 22: BGP peer flap recovery result

16.4 Observation

When the direct BGP peer between `hpe-r1` and `hpe-r9` was disabled, `hpe-r1` quickly selected the alternate BGP route through `hpe-r2`. The route switched to the alternate next-hop in 93 ms.

After the BGP peer was re-enabled, the session was re-established in 1596 ms, and the route switched back to the direct next-hop in 1754 ms.

16.5 Conclusion

This experiment proves that the BGP design supports peer flap recovery and automatic alternate path selection. Traffic continued with 0% packet loss during the flap, showing strong forwarding continuity during BGP peer-level failure.

17 LLGR Stale Route Timer Experiment

17.1 Purpose

This experiment was performed to observe stale-route handling and withdrawal timer behaviour with Long-Lived Graceful Restart. The goal was to check whether a route learned from a failed BGP peer remains available temporarily and is removed after the configured timer period.

17.2 Experiment Setup

The route tested was:

10.0.93.0/24

This prefix is originated from the ISP/upstream side behind **hpe-r9**. The observing router was **hpe-r1**.

For the experiment, the timers were temporarily reduced:

Timer	Temporary Value
Graceful Restart time	5 seconds
LLGR stale time	10 seconds
Expected total retention window	About 15 seconds

Table 23: Temporary GR and LLGR timer values

After applying the temporary timer configuration, the BIRD daemon on **hpe-r9** was stopped. The route table on **hpe-r1** was then sampled repeatedly to check when the route disappeared.

17.3 Measurement Result

Measurement	Result
Target prefix	10.0.93.0/24
GR timer	5 seconds
LLGR stale timer	10 seconds
Last time route from hpe-r9 was present	14259 ms
First time route from hpe-r9 became missing	15350 ms
LLGR stale marker observed	Yes, shown as (100s) and BGP.community: (65535,6)

Table 24: LLGR stale route timer result

17.4 Observation

The route did not disappear immediately after the peer-side failure. Instead, it remained present for approximately 14.259 seconds and disappeared around 15.350 seconds.

This closely matches the configured timing expectation of 5 seconds of Graceful Restart time plus 10 seconds of LLGR stale time.

The literal text `LLGR_STALE` was not printed by BIRD. However, the stale route marker was visible as `(100s)` and the BGP community `(65535,6)`. The community `65535:6` corresponds to the LLGR stale community. Therefore, this experiment confirms both stale-route retention timing and visible LLGR stale marking.

17.5 Conclusion

This experiment demonstrates stale-route retention and withdrawal timer behaviour. With shortened GR and LLGR timers, the route remained temporarily available, was marked stale using `(100s)` and `BGP.community: (65535,6)`, and was removed after approximately the expected timer duration. This satisfies the stale-route handling and withdrawal-timer observation requirement.

18 Packet Capture Validation

18.1 Purpose

This experiment validates the routing-control behaviour using packet capture. Earlier experiments measured convergence using BIRD state, Linux route-table changes, CSV logs, and ping results. Packet capture adds protocol-level evidence by showing that BFD and BGP packets were actually observed during the experiments.

18.2 Captured Protocols

Capture	Filter	Purpose
Multi-hop BFD	udp port 4784 or udp port 3784	To verify that BFD control packets were exchanged.
BGP peer flap	tcp port 179	To verify BGP control-plane traffic during peer disable and re-enable.

Table 25: Packet capture filters used

18.3 Measurement Result

Measurement	Result
BFD pcap file	evidence/packet_capture/multihop_bfd_20260624_013648.pcap
BGP pcap file	evidence/packet_capture/bgp_peer_flap_20260624_013648.pcap
BFD packet count	347
BGP packet count	36
BFD summary file	evidence/packet_capture/multihop_bfd_summary_20260624_013648.txt
BGP summary file	evidence/packet_capture/bgp_peer_flap_summary_20260624_013648.txt

Table 26: Packet capture result summary

18.4 Observation

The BFD capture recorded 347 packets, confirming that BFD control traffic was present. The BGP capture recorded 36 packets during the peer flap, confirming that BGP control-plane traffic over TCP port 179 was observed when the BGP peer was disabled and re-enabled.

18.5 Conclusion

This experiment confirms that packet-level evidence was collected for the routing mechanisms. The generated pcap files can be opened in Wireshark or inspected using tcpdump/tshark for deeper protocol-level analysis.

19 Result Graphs

19.1 Purpose

This section presents graphical summaries of the major experimental results. The graphs make it easier to compare convergence time, packet loss, and additional validation evidence across the different routing mechanisms.

19.2 Convergence and Reaction Time Comparison

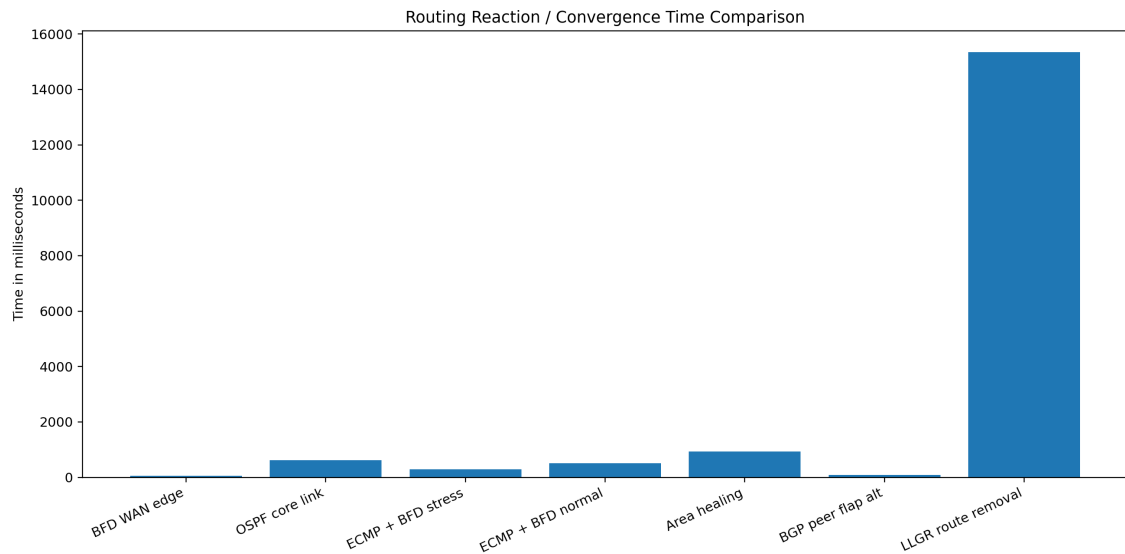


Figure 2: Routing reaction and convergence time comparison

This graph compares the measured reaction or convergence time across different experiments. Lower values indicate faster failure detection or faster route recovery. The BFD WAN edge failure and BGP peer flap alternate-path switch showed very fast reaction times.

19.3 Packet Loss Comparison

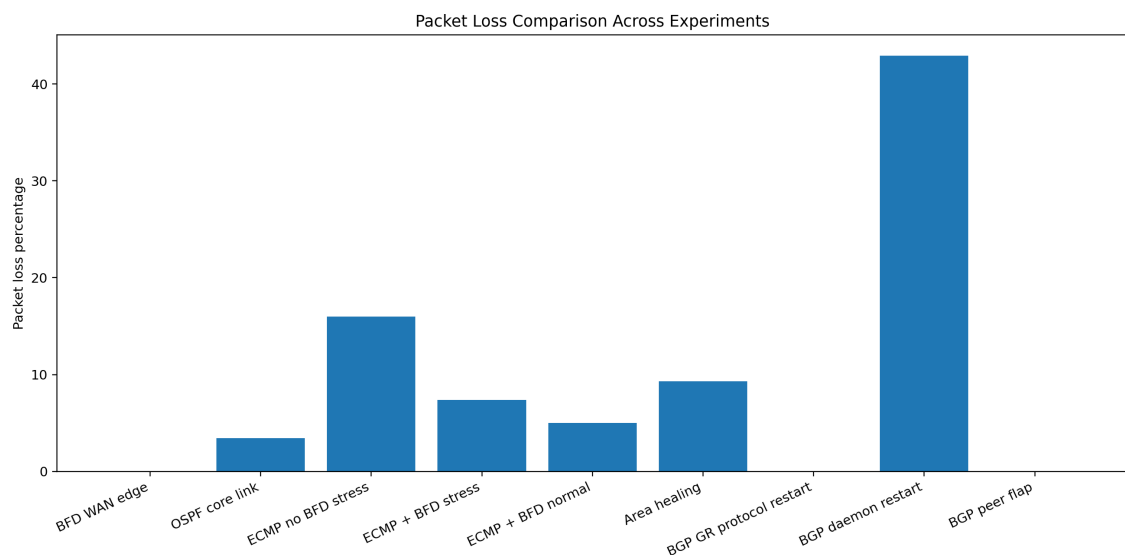


Figure 3: Packet loss comparison across experiments

This graph compares the traffic impact during each experiment. The BFD WAN edge test, BGP GR protocol restart test, and BGP peer flap test preserved traffic with zero

packet loss. The full BIRD daemon restart caused higher loss and is therefore reported as a limitation.

19.4 Additional Deliverable Evidence

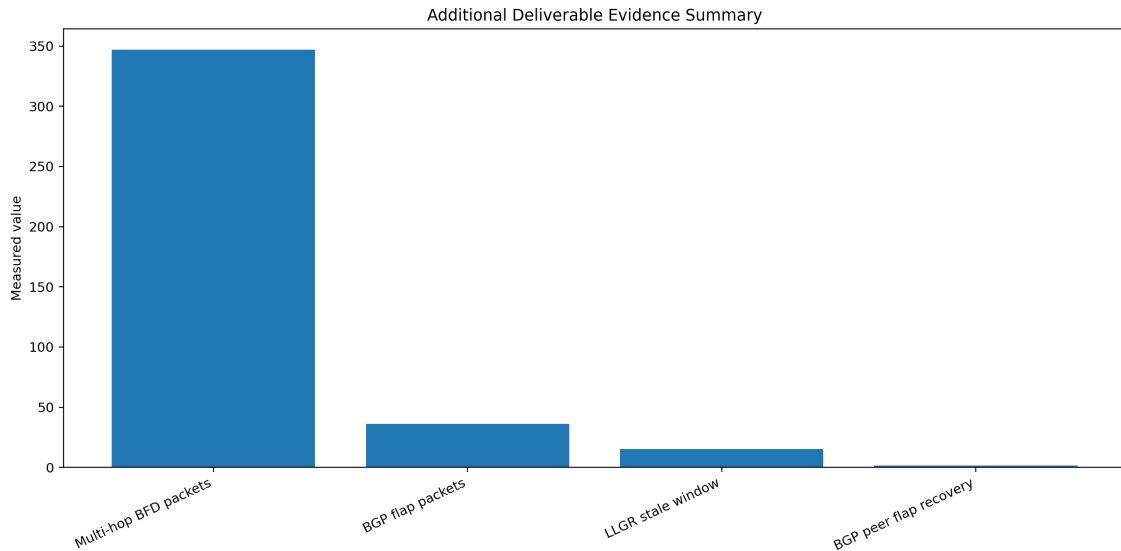


Figure 4: Additional deliverable evidence summary

This graph summarizes extra deliverables such as packet capture counts, LLGR stale-route timing, and BGP peer flap recovery. These results strengthen the project by adding packet-level validation and stale-route timer observation.

20 Final Lab Validation

After all failure experiments, configuration changes, BGP withdrawals, and recovery tests, a final validation was performed.

The purpose was to verify that the lab returned to a healthy state.

Test	Source	Target	TX	RX	Loss	Status
h1_to_h2	hpe-h1	10.0.82.2	5	5	0%	pass
h2_to_h1	hpe-h2	10.0.61.2	5	5	0%	pass
h1_to_h3	hpe-h1	10.0.93.2	5	5	0%	pass
h3_to_h1	hpe-h3	10.0.61.2	5	5	0%	pass
h2_to_h3	hpe-h2	10.0.93.2	5	5	0%	pass
h3_to_h2	hpe-h3	10.0.82.2	5	5	0%	pass

Table 27: Final end-to-end connectivity validation

All final connectivity tests passed with 0% packet loss. This confirms that the routing lab was restored successfully after all experiments.

21 Summary of Measured Results

Mechanism	Measured Result	Conclusion
BFD WAN edge failure	61 ms detection and 0% packet loss	Fast sub-second failure detection achieved.
OSPF core link failure	Kernel/BIRD route update around 628–648 ms	OSPF recovered within sub-second range.
OSPF ECMP native	840 ms kernel monitor, 16% packet loss under stress	ECMP recovery worked but with higher loss.
OSPF ECMP with BFD	292 ms under stress, 517 ms under normal traffic	BFD improved ECMP failover behaviour.
NSSA Type-7/Type-5	Type-7 inside NSSA, Type-5 outside NSSA	NSSA translation and containment worked.
OSPF area healing	BIRD/kernel update around 931–962 ms	Inter-area healing across ABRs worked.
BGP GR forwarding	200/200 packets received, 0% loss	Forwarding preserved during BGP restart.
BGP withdrawal	Route missing in 134 ms, 95% loss during test	True route withdrawal removed forwarding path.
Final validation	All six end-to-end tests passed	Lab restored to healthy state.

22 To be Completed

- A dedicated multi-hop BFD experiment can be added.
- Full LLGR stale timer expiry can be observed by waiting for the configured LL stale time.
- A full BIRD daemon restart experiment can be added. The current test focused on BGP protocol/control-plane restart.
- Graphs can be generated from the CSV files to improve visual analysis.
- tcpdump or tshark packet captures can be added to show control-plane packet behaviour.
- BIRD source-code patching can be considered as an optional future extension.

23 Conclusion

This project successfully implemented a self-driving dynamic routing lab using OSPF and BGP in the BIRD routing stack.

The lab demonstrated fast failure detection using BFD, OSPF recovery under core link failures, ECMP failover behaviour, NSSA Type-7 to Type-5 translation, stub-area containment, OSPF area healing across ABRs, and BGP Graceful Restart forwarding continuity.

The strongest BFD result showed a 61 ms failure detection time. OSPF recovery and area healing were observed within approximately one second. ECMP failover improved when OSPF-BFD was enabled. BGP Graceful Restart preserved forwarding during a control-plane restart with 0% packet loss, while a true BGP route withdrawal correctly removed the route and caused traffic failure.

The final validation confirmed that all end-to-end traffic paths were restored successfully after the experiments.

Overall, the project satisfies the main proposal requirements and demonstrates how modern networks can automatically detect failures, preserve forwarding, and recover without manual intervention.

24 Screenshot Evidence Appendix

This appendix contains selected screenshots captured during the experiments. These screenshots support the measurements and conclusions discussed in the report.

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/bfd/bfd_edge_failure.csv
timestamp,test_name,failed_router,failed_interface,peer_ip,bfd_detection_ms,bgp_reaction_ms
,estimated_tx,received,lost,loss_percent,explicit_unreachable_packets,fail_ms,restore_ms
20260623_145246,bfd_wan_edge_failure,hpe-r2,eth1,10.0.29.3,61,61,293,293,0,0.00,0,178220657
2526,1782206576853
```

Figure 5: BFD detection result CSV

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/bfd/bfd_alterate_path_verification_*.txt | head -1)
grep -A 25 "3. During failure path check" "$LATEST"
3. During failure path check
=====

---- hpe-r2 route to hpe-h3 during failure ----
10.0.93.2 via 10.0.12.2 dev eth2 src 10.0.12.3 uid 0
    cache
BIRD 2.0.8 ready.
Table master4:
10.0.93.0/24      unicast [r1 08:45:06.647] * (100) [AS65002i]
    via 10.0.12.2 on eth2
    Type: BGP univ
    BGP.origin: IGP
    BGP.as_path: 65002
    BGP.next_hop: 10.0.12.2
    BGP.local_pref: 100

---- hpe-r1 route to hpe-h3 during failure ----
10.0.93.2 via 10.0.19.3 dev eth0 src 10.0.19.2 uid 0
    cache
BIRD 2.0.8 ready.
Table master4:
10.0.93.0/24      unicast [r9 08:45:06.646] * (100) [AS65002i]
    via 10.0.19.3 on eth0
    Type: BGP univ
    BGP.origin: IGP
    BGP.as_path: 65002
```

Figure 6: BFD alternate path during failure

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/bfd/bfd_restore_confirmation_*.txt | head -1)
grep -E "BFD|Up|BGP|Established|0% packet loss" "$LATEST"
BFD Restore Confirmation
3 packets transmitted, 3 received, 0% packet loss, time 2085ms
3 packets transmitted, 3 received, 0% packet loss, time 2060ms
===== hpe-r2 BFD sessions =====
10.0.29.3          eth1          Up           09:25:47.873   0.100   0.500
===== hpe-r9 BFD sessions =====
10.0.19.2          eth0          Up           08:45:02.355   0.100   0.500
10.0.29.2          eth1          Up           09:25:47.873   0.100   0.500
===== hpe-r2 BGP protocols =====
r9      BGP      ---      up      09:27:32.316  Established
r1      BGP      ---      up      08:45:05.679  Established
===== hpe-r9 BGP protocols =====
r1      BGP      ---      up      08:45:06.408  Established
r2      BGP      ---      up      09:27:32.316  Established
5 packets transmitted, 5 received, 0% packet loss, time 4093ms

```

Figure 7: BFD restore confirmation

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/ospf/ospf_core_link_failure.csv
timestamp,test_name,old_next_hop,new_next_hop,kernel_monitor_ms,route_get_sample_ms,bird_ro
ute_sample_ms,ping_tx_estimated,ping_rx,failed_or_lost_packets,ping_loss_percent,explicit_u
nreachable_packets,note
20260623_151142,ospf_r3_r2_core_link_failure,10.0.23.2,10.0.13.2,628,631,648,350,338,12,3.4
3,1,ping loss estimated from ICMP sequence numbers because normal ping summary was unavaila
ble

```

Figure 8: OSPF core failure CSV

```

1782207709010 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709037 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709064 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709091 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709117 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709143 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709168 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709193 10.0.93.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
1782207709218 RTNETLINK answers: Network is unreachable
1782207709243 RTNETLINK answers: Network is unreachable
1782207709269 RTNETLINK answers: Network is unreachable
1782207709295 RTNETLINK answers: Network is unreachable
1782207709322 RTNETLINK answers: Network is unreachable
1782207709346 RTNETLINK answers: Network is unreachable
1782207709372 RTNETLINK answers: Network is unreachable
1782207709398 RTNETLINK answers: Network is unreachable
1782207709423 RTNETLINK answers: Network is unreachable
1782207709449 RTNETLINK answers: Network is unreachable
1782207709473 RTNETLINK answers: Network is unreachable
1782207709498 RTNETLINK answers: Network is unreachable
1782207709525 RTNETLINK answers: Network is unreachable
1782207709551 RTNETLINK answers: Network is unreachable
1782207709575 RTNETLINK answers: Network is unreachable
1782207709602 RTNETLINK answers: Network is unreachable
1782207709629 RTNETLINK answers: Network is unreachable
1782207709656 RTNETLINK answers: Network is unreachable
1782207709683 RTNETLINK answers: Network is unreachable
1782207709710 RTNETLINK answers: Network is unreachable
1782207709737 RTNETLINK answers: Network is unreachable
1782207709764 RTNETLINK answers: Network is unreachable
1782207709791 RTNETLINK answers: Network is unreachable
1782207709817 RTNETLINK answers: Network is unreachable
1782207709844 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207709869 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207709894 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207709921 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207709946 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207709971 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207709995 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207710021 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207710048 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207710075 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
1782207710102 10.0.93.2 via 10.0.13.2 dev eth2 src 10.0.13.3 uid 0
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ =|

```

Figure 9: OSPF route sampling during recovery


```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
for r in hpe-r1 hpe-r2 hpe-r3 hpe-r4 hpe-r5 hpe-r6 hpe-r7 hpe-r8; do
  echo "$r"
  docker exec "$r" birdc show protocols | grep -E "ospf|Running"
done
[hpe-r1]
ospf1      OSPF      master4    up        09:18:25.651  Running
[hpe-r2]
ospf1      OSPF      master4    up        09:18:25.696  Running
[hpe-r3]
ospf1      OSPF      master4    up        09:18:25.739  Running
[hpe-r4]
ospf1      OSPF      master4    up        09:18:25.794  Running
[hpe-r5]
ospf1      OSPF      master4    up        09:18:25.845  Running
[hpe-r6]
ospf1      OSPF      master4    up        09:18:25.895  Running
[hpe-r7]
ospf1      OSPF      master4    up        09:18:25.938  Running
[hpe-r8]
ospf1      OSPF      master4    up        09:18:25.983  Running

```

Figure 10: OSPF final health

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/ospf/ospf_ecmp_comparison.csv
test,ospf_bfd,ping_interval,kernel_monitor_ms,kernel_route_sample_ms,route_get_sample_ms,
bird_route_sample_ms,tx,rx,lost,loss_percent,interpretation
native_ospf_ecmp_without_bfd,no,0.02s,840,820,94,819,500,420,80,16.00,native OSPF ECMP fa
ilover baseline
ospf_ecmp_with_bfd_stress,yes,0.02s,292,305,49,304,500,463,37,7.40,BFD-assisted ECMP fail
over under high-rate stress traffic
ospf_ecmp_with_bfd_normal,yes,1s,517,542,63,540,20,19,1,5.00,BFD-assisted ECMP failover u
nder normal-rate traffic

```

Figure 11: OSPF ECMP comparison CSV

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
docker exec hpe-r3 birdc show route 10.0.24.0/24 all
docker exec hpe-r3 ip route show 10.0.24.0/24
BIRD 2.0.8 ready.
Table master4:
10.0.24.0/24          unicast [ospf1 10:26:00.294] * I (150/20) [4.4.4.4]
    via 10.0.23.2 on eth3 weight 1
    via 10.0.34.3 on eth0 weight 1
    Type: OSPF univ
    OSPF.metric1: 20
    OSPF.router_id: 4.4.4.4
10.0.24.0/24 proto bird metric 32
    nexthop via 10.0.23.2 dev eth3 weight 1
    nexthop via 10.0.34.3 dev eth0 weight 1

```

Figure 12: OSPF ECMP route before failure

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

echo "---- hpe-r3 BFD ----"
docker exec hpe-r3 birdc show bfd sessions

echo
echo "---- hpe-r3 OSPF neighbours ----"
docker exec hpe-r3 birdc show ospf neighbors
---- hpe-r3 BFD ----
BIRD 2.0.8 ready.
bfd1:


| IP address | Interface | State | Since        | Interval | Timeout |
|------------|-----------|-------|--------------|----------|---------|
| 10.0.35.3  | eth1      | Up    | 10:16:30.905 | 0.100    | 0.500   |
| 10.0.34.3  | eth0      | Up    | 10:16:30.034 | 0.100    | 0.500   |
| 10.0.23.2  | eth3      | Up    | 10:25:55.698 | 0.100    | 0.500   |
| 10.0.13.2  | eth2      | Up    | 10:16:29.865 | 0.100    | 0.500   |



---- hpe-r3 OSPF neighbours ----
BIRD 2.0.8 ready.
ospf1:


| Router ID | Pri | State    | DTime  | Interface | Router IP |
|-----------|-----|----------|--------|-----------|-----------|
| 4.4.4.4   | 1   | Full/DR  | 16.260 | eth0      | 10.0.34.3 |
| 5.5.5.5   | 1   | Full/DR  | 16.311 | eth1      | 10.0.35.3 |
| 1.1.1.1   | 1   | Full/BDR | 16.117 | eth2      | 10.0.13.2 |
| 2.2.2.2   | 1   | Full/DR  | 16.162 | eth3      | 10.0.23.2 |


```

Figure 13: OSPF-BFD sessions active

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/nssa/nssa_static_external_route.csv
timestamp,router,route_present,first_route_type,first_route_line
20260623_162347,hpe-r6,yes,Type: static univ,172.16.66.0/24 blackhole [nssa_test_static 10:53:47.630] * (200)
20260623_162347,hpe-r5,yes,Type: OSPF-E2 univ,172.16.66.0/24 unicast [ospf1 10:53:53.217] * E2 (150/20/66) [6.6.6.6]
20260623_162347,hpe-r3,yes,Type: OSPF-E2 univ,172.16.66.0/24 unicast [ospf1 10:53:52.294] * E2 (150/30/66) [6.6.6.6]
20260623_162347,hpe-r1,yes,Type: OSPF-E2 univ,172.16.66.0/24 unicast [ospf1 10:53:52.420] * E2 (150/40/66) [3.3.3.3]
20260623_162347,hpe-r2,yes,Type: OSPF-E2 univ,172.16.66.0/24 unicast [ospf1 10:53:52.846] * E2 (150/40/66) [3.3.3.3]
20260623_162347,hpe-r4,yes,Type: OSPF-E2 univ,172.16.66.0/24 unicast [ospf1 10:53:52.754] * E2 (150/40/66) [3.3.3.3]
20260623_162347,hpe-r8,no,unknown,

```

Figure 14: NSSA Type-7 and Type-5 CSV

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

for r in hpe-r6 hpe-r5 hpe-r3; do
echo
echo "===== $r LSADB type 7 ====="
docker exec "$r" birdc show ospf lsadb type 7 | grep -E "Type|172.16.66.255|0007"
done

===== hpe-r6 LSADB type 7 =====


| Type | LS ID         | Router  | Sequence | Age | Checksum |
|------|---------------|---------|----------|-----|----------|
| 0007 | 172.16.66.255 | 6.6.6.6 | 80000002 | 277 | a283     |



===== hpe-r5 LSADB type 7 =====


| Type | LS ID         | Router  | Sequence | Age | Checksum |
|------|---------------|---------|----------|-----|----------|
| 0007 | 172.16.66.255 | 6.6.6.6 | 80000002 | 276 | a283     |



===== hpe-r3 LSADB type 7 =====


| Type | LS ID         | Router  | Sequence | Age | Checksum |
|------|---------------|---------|----------|-----|----------|
| 0007 | 172.16.66.255 | 6.6.6.6 | 80000002 | 278 | a283     |


```

Figure 15: Type-7 LSA inside NSSA

```

0007 172.16.66.255 6.6.6.6 80000002 278 a283
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

for r in hpe-r3 hpe-r1 hpe-r2 hpe-r4; do
    echo
    echo "===== $r LSADB type 5 ====="
    docker exec "$r" birdc show ospf lsadb type 5 | grep -E "Type|172.16.66.255|0005"
done

===== hpe-r3 LSADB type 5 =====
Type LS ID Router Sequence Age Checksum
0005 0.0.0.0 1.1.1.1 80000004 604 239e
0005 0.0.0.0 2.2.2.2 80000006 164 01ba
0005 172.16.66.255 3.3.3.3 80000001 278 75c5

===== hpe-r1 LSADB type 5 =====
Type LS ID Router Sequence Age Checksum
0005 0.0.0.0 1.1.1.1 80000004 604 239e
0005 0.0.0.0 2.2.2.2 80000006 164 01ba
0005 172.16.66.255 3.3.3.3 80000001 280 75c5

===== hpe-r2 LSADB type 5 =====
Type LS ID Router Sequence Age Checksum
0005 0.0.0.0 1.1.1.1 80000004 604 239e
0005 0.0.0.0 2.2.2.2 80000006 163 01ba
0005 172.16.66.255 3.3.3.3 80000001 280 75c5

===== hpe-r4 LSADB type 5 =====
Type LS ID Router Sequence Age Checksum
0005 0.0.0.0 1.1.1.1 80000004 604 239e
0005 0.0.0.0 2.2.2.2 80000006 164 01ba
0005 172.16.66.255 3.3.3.3 80000001 280 75c5
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ |

```

Figure 16: Type-5 LSA outside NSSA

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

echo "hpe-r8 does not have the external route in BIRD:"
docker exec hpe-r8 birdc show route 172.16.66.0/24 all

echo
echo "hpe-r8 still has a default path for forwarding:"
docker exec hpe-r8 ip route get 172.16.66.1
hpe-r8 does not have the external route in BIRD:
BIRD 2.0.8 ready.
Network not found

hpe-r8 still has a default path for forwarding:
172.16.66.1 via 10.0.78.2 dev eth0 src 10.0.78.3 uid 0
cache

```

Figure 17: Stub area external route containment

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/area_healing/ospf_area_healing_r3_r4_failure.csv
timestamp,test_name,failed_link,old_next_hop,route_get_ms,kernel_route_ms,bird_route_ms,ping
_tx,ping_rx,ping_lost,ping_loss_percent,route_get_first_line,kernel_first_line,bird_first_li
ne,missing_ping_sequences
20260623_163945,ospf_area_healing_r3_r4,hpe-r3_eth0,10.0.34.3,80,931,962,150,136,14,9.33333,
"10.0.82.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0","10.0.82.0/24 proto bird metric 32
    nexthop via 10.0.13.2 dev eth2 weight 1          nexthop via 10.0.23.2 dev eth3 weight
1","10.0.82.0/24          unicast [ospf1 11:09:52.294] * IA (150/50) [4.4.4.4]          via 10
.0.13.2 on eth2 weight 1          via 10.0.23.2 on eth3 weight 1","11 12 13 14 15 16 17 18 19 20
21 22 23 24"

```

Figure 18: OSPF area healing CSV

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/area_healing/area_healing_r3_r4_failure_*.txt | head -1)

grep -E "Route-get first switched line|Kernel first switched line|BIRD first switched line"
"$LATEST"
Route-get first switched line: 10.0.82.2 via 10.0.23.2 dev eth3 src 10.0.23.3 uid 0
Kernel first switched line: 10.0.82.0/24 proto bird metric 32  nexthop via 10.0.13.2 dev et
h2 weight 1  nexthop via 10.0.23.2 dev eth3 weight 1
BIRD first switched line: 10.0.82.0/24          unicast [ospf1 11:09:52.294] * IA (150/50) [4
.4.4.4]          via 10.0.13.2 on eth2 weight 1  via 10.0.23.2 on eth3 weight 1

```

Figure 19: OSPF area route after healing

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/area_healing/area_healing_r3_r4_failure_*.txt | head -1)

grep -A 12 "Final ping hpe-h1 to hpe-h2" "$LATEST"
---- Final ping hpe-h1 to hpe-h2 ----
PING 10.0.82.2 (10.0.82.2) 56(84) bytes of data.
64 bytes from 10.0.82.2: icmp_seq=1 ttl=58 time=0.168 ms
64 bytes from 10.0.82.2: icmp_seq=2 ttl=58 time=0.195 ms
64 bytes from 10.0.82.2: icmp_seq=3 ttl=58 time=0.304 ms
64 bytes from 10.0.82.2: icmp_seq=4 ttl=58 time=0.347 ms
64 bytes from 10.0.82.2: icmp_seq=5 ttl=58 time=0.236 ms

--- 10.0.82.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4102ms
rtt min/avg/max/mdev = 0.168/0.250/0.347/0.066 ms
=====

```

Figure 20: OSPF area final restored health

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/bgp_gr_llgr/bgp_gr_forwarding_blackhole_guard.csv
timestamp,test_name,blackhole_guard_enabled,r1_r2_link_down,old_next_hop,bgp_non_established
_seen_ms,old_next_hop_seen_after_restart,alternate_next_hop_seen_after_restart,blackhole_or_
unreachable_seen_after_restart,bird_route_missing_seen_after_restart,stale_or_grace_text_see
n,ping_tx,ping_rx,ping_lost,ping_loss_percent,bgp_non_established_line,missing_ping_sequence
s
20260623_170932,bgp_gr_blackhole_guard,yes,yes,10.0.19.3,114,yes,no,no,yes,no,200,200,0,0,"N
ame Proto Table State Since Info r9 BGP --- s
tart 11:39:44.594 Active Received: Administrative reset","

```

Figure 21: BGP GR blackhole guard CSV

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

LATEST=$(ls -t evidence/bgp_gr_llgr/bgp_gr_blackhole_guard_*.txt | head -1)

grep -E "Add blackhole|10.0.93.0/24 via 10.0.19.3|blackhole 10.0.93.0/24|Route-get should|10.0.93.2 via 10.0.19.3" "$LATEST"
1. Add blackhole guard route on hpe-r1
10.0.93.0/24 via 10.0.19.3 dev eth0 proto bird metric 32
blackhole 10.0.93.0/24 metric 9999
Route-get should still prefer real BGP route via 10.0.19.3:
10.0.93.2 via 10.0.19.3 dev eth0 src 10.0.19.2 uid 0
10.0.93.2 via 10.0.19.3 dev eth0 src 10.0.19.2 uid 0
10.0.93.2 via 10.0.19.3 dev eth0 src 10.0.19.2 uid 0

```

Figure 22: BGP GR blackhole guard real route preferred

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

LATEST=$(ls -t evidence/bgp_gr_llgr/bgp_gr_blackhole_guard_*.txt | head -1)

grep -E "Bringing down hpe-r1 eth1|eth1.*DOWN|hpe-r1 BGP protocols after isolation|r2|r9" "$LATEST" | head -40
2. Disable alternate hpe-r1 <-> hpe-r2 path
Disabling BGP protocol r2 on hpe-r1...
r2: disabled
Bringing down hpe-r1 eth1, the r1-r2 link...
eth1@if101 DOWN 10.0.12.2/24
hpe-r1 BGP protocols after isolation:
r9 BGP --- up 11:35:53.637 Established
r2 BGP --- down 11:39:32.942
10.0.93.0/24 unicast [r9 11:35:53.678] * (100) [AS65002i]
Baseline ping under blackhole guard + no r2 link:
4. Restart direct BGP peer session hpe-r9 -> hpe-r1
Running: docker exec hpe-r9 birdc restart r1
10.0.93.0/24 unicast [r9 11:39:48.878] * (100) [AS65002i]
BGP non-established line: Name Proto Table State Since Info r9
BGP --- start 11:39:44.594 Active Received: Administrative reset

```

Figure 23: BGP alternate path disabled

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

LATEST=$(ls -t evidence/bgp_gr_llgr/bgp_gr_blackhole_guard_*.txt | head -1)

grep -E "Restart time|Running:|BGP non-established seen ms|BGP non-established line|Active|Administrative reset" "$LATEST"
Restart time relative to monitor start: 1007 ms
Running: docker exec hpe-r9 birdc restart r1
BGP non-established seen ms: 114
BGP non-established line: Name Proto Table State Since Info r9
BGP --- start 11:39:44.594 Active Received: Administrative reset

```

Figure 24: BGP restart observed

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/bgp_gr_llgr/bgp_gr_blackhole_guard_*.txt | head -1)
grep -E "Old next-hop seen after restart|Alternate next-hop seen after restart|Blackhole/unreachable seen after restart|Ping transmitted|Ping received|Ping lost|Ping loss percent|Missing ping sequences" "$LATEST"
Old next-hop seen after restart: yes
Alternate next-hop seen after restart: no
Blackhole/unreachable seen after restart: no
Ping transmitted: 200
Ping received: 200
Ping lost: 0
Ping loss percent: 0
Missing ping sequences:

```

Figure 25: BGP forwarding continued without fallback

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/bgp_gr_llgr/bgp_route_withdrawal_final.csv
timestamp,test_name,blackhole_guard_enabled,r1_r2_path_disabled,withdrawn_prefix,bgp_session_non_established_seen_after_withdraw,old_next_hop_seen_after_withdraw,alternate_next_hop_seen_after_withdraw,blackhole_seen_ms,bird_route_missing_ms,ping_tx,ping_rx,ping_lost,ping_loss_percent,blackhole_line,bird_route_missing_line,missing_ping_sequences
20260623_172052,bgp_route_withdrawal,yes,yes,10.0.93.0/24,no,yes,no,NA,134,220,11,209,95,"not observed","Network not found",""

```

Figure 26: BGP withdrawal CSV

```

10.0.93.2 vld 10.0.19.5 dev eth0 src 10.0.19.2 dst 0
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/bgp_gr_llgr/bgp_route_withdrawal_*.txt | head -1)
grep -E "BGP non-established seen after withdrawal|BIRD route missing ms|BIRD route missing line|Network not found" "$LATEST"
Network not found
BGP non-established seen after withdrawal: no
BIRD route missing ms: 134
BIRD route missing line: Network not found
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

```

Figure 27: BGP route withdrawn while session stayed up

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
LATEST=$(ls -t evidence/bgp_gr_llgr/bgp_route_withdrawal_*.txt | head -1)
grep -E "Final ping after withdrawal|packets transmitted|Ping transmitted|Ping received|Ping lost|Ping loss percent" "$LATEST"
5 packets transmitted, 5 received, 0% packet loss, time 4117ms
Final ping after withdrawal:
5 packets transmitted, 0 received, 100% packet loss, time 4133ms
Ping transmitted: 220
Ping received: 11
Ping lost: 209
Ping loss percent: 95

```

Figure 28: BGP withdrawal packet loss

24.1 Additional Deliverable Terminal Evidence

The following screenshots show terminal evidence for the additional deliverable experiments: multi-hop BFD, BGP full daemon restart, and BGP peer flap recovery.

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird
cat results/multihop_bfd/multihop_bfd_failure.csv
timestamp,test_name,failed_link,r1_peer,r8_peer,r1_bfd_detection_ms,r8_bfd_detection_ms,r1_
recovery_ms,r8_recovery_ms,ping_tx,ping_rx,ping_loss,target_under_1s
20260623_230920,multihop_bfd_failure,hpe-r4-eth2_to_hpe-r7-eth1,10.0.82.3,10.0.14.2,546,629
,7645,2,269,79,70.632%,yes
```

Figure 29: 01 multihop bfd sessions

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ docker exec hpe-r1 birdc show bfd
sessions
docker exec hpe-r8 birdc show bfd sessions
BIRD 2.0.8 ready.
bfd1:
IP address      Interface State      Since          Interval  Timeout
10.0.82.3        ---      Up         17:39:32.615   0.100    0.500
10.0.19.3        eth0     Up         18:48:38.580   0.100    0.500
10.0.12.3        eth1     Up         11:51:35.751   0.100    0.500
10.0.13.3        eth2     Up         10:16:29.865   0.100    0.500
10.0.14.3        eth3     Up         10:16:29.878   0.100    0.500
BIRD 2.0.8 ready.
bfd1:
IP address      Interface State      Since          Interval  Timeout
10.0.14.2        ---      Up         17:39:32.615   0.100    0.500
10.0.78.2        eth0     Up         10:16:31.862   0.100    0.500
```

Figure 30: 02 multihop bfd result csv

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cat results/bgp_daemon_restart/bg
p_gr_daemon_restart.csv
timestamp,test_name,restarted_router,birdc_ready_ms,bgp_reestablished_ms,ping_tx,ping_rx,pi
ng_loss,direct_next_hop_samples,alternate_next_hop_samples,route_missing_samples,forwarding
_preserved
20260623_235615,bgp_gr_full_bird_daemon_restart,hpe-r9,187,4761,174,96,44.8276%,44,0,0,part
ial_or_no
```

Figure 31: 03 bgp daemon restart result csv

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cat results/bgp_peer_flap/bgp_pee
r_flap.csv
timestamp,test_name,flapped_router,flapped_peer,target,route_to_alternate_ms,bgp_reestablis
hed_ms,route_to_direct_ms,ping_tx,ping_rx,ping_loss
20260624_001833,bgp_peer_flap,hpe-r1,r9,10.0.93.2,93,1596,1754,162,162,0%
```

Figure 32: 04 bgp peer flap result csv

```

gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird/report$ cd ~/Documents/HPE_bird

grep "BGP.community: (65535,6)" evidence/llgr_stale_timer/llgr_route_samples_20260624_011352.log | head -3
5522 ms | r9_route_present=yes | stale_text_seen=no | BIRD 2.0.8 ready. Table master4: 10.0.93.0/24 unicast
[r9 19:44:03.555] * (100s) [AS65002i] via 10.0.19.3 on eth0 Type: BGP univ BGP.origin: IGP
BGP.as_path: 65002 BGP.next_hop: 10.0.19.3 BGP.local_pref: 100 BGP.community: (65535,6)

6618 ms | r9_route_present=yes | stale_text_seen=no | BIRD 2.0.8 ready. Table master4: 10.0.93.0/24 unicast
[r9 19:44:03.556] * (100s) [AS65002i] via 10.0.19.3 on eth0 Type: BGP univ BGP.origin: IGP
BGP.as_path: 65002 BGP.next_hop: 10.0.19.3 BGP.local_pref: 100 BGP.community: (65535,6)

7703 ms | r9_route_present=yes | stale_text_seen=no | BIRD 2.0.8 ready. Table master4: 10.0.93.0/24 unicast
[r9 19:44:03.555] * (100s) [AS65002i] via 10.0.19.3 on eth0 Type: BGP univ BGP.origin: IGP
BGP.as_path: 65002 BGP.next_hop: 10.0.19.3 BGP.local_pref: 100 BGP.community: (65535,6)

```

Figure 33: LLGR stale route marker shown as (100s) and BGP.community: (65535,6)